Save Point

AI Providers

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

The app is bring-your-own-API-key: no AI usage is billed to the app itself. All AI calls go through the `AIProvider` abstraction in `lib/ai/base.dart`, so the rest of the codebase never talks to a specific vendor's API shape directly.

Why plain `http` instead of vendor SDKs

`lib/ai/openai_provider.dart` and `anthropic_provider.dart` both use Dart's plain `http` package directly rather than any official vendor SDK:

- One HTTP dependency instead of two (or more) heavier SDKs.
- Full control over `baseUrl` overrides (see below); both providers stay structurally identical this way.
- Raw HTTP is far easier to mock deterministically in tests than SDK internals. `test/ai/openai_provider_test.dart` and `anthropic_provider_test.dart` use `http`'s own `testing.dart` (`MockClient`) to assert on exact requests and simulate arbitrary error responses, with no real network call or API key involved.

`AIProvider` (base.dart)

```
abstract class AIProvider {
  AIProvider({
    required this.apiKey,
    required this.model,
    this.baseUrl,
    this.timeout = const Duration(seconds: 30),
    http.Client? httpClient,
  });

  Future<List<GeneratedQuestion>> generateQuestions(
    String noteText, {
    int numQuestions = 5,
    String questionFormat = questionTypeShortAnswer,
    List<LinkedNoteContext>? linkedNotesContext,
  });
}
```

Every provider takes an injectable `http.Client` (defaulting to a real one), which is exactly what makes the mocked tests above possible without any production code path needing to know it's being tested.

Only one method. Short-answer grading is *not* part of this interface; grading is local text/keyword matching only (`lib/core/grading.dart`), a deliberate product decision, not an oversight (same as the Python desktop app this was ported from).

`questionFormat` is always `'short_answer'` or `'multiple_choice'` ; the caller decides this ahead of time and instructs the AI to generate only that format. `linkedNotesContext` exists in the interface (mirroring the Python app's shape) but is always empty right now, since automatic note-linking hasn't been ported over yet. See `future_work.md` .

Both provider implementations:

1. Build a `(systemPrompt, userPrompt)` pair via `lib/ai/prompts.dart` , which instructs the model to respond with strict JSON only (no prose, no markdown fencing) matching a fixed schema.
2. POST to the provider's endpoint.
3. Parse the JSON text response via the shared `parseGeneratedQuestions()` helper in `base.dart` .
4. Wrap any network failure, non-2xx HTTP status, invalid JSON, or unexpected response shape into a single `AIProviderError` . Callers (the note editor's "Generate" button) only ever need to catch this one type.

`OpenAIProvider`

OpenAI-compatible Chat Completions shape: `POST {baseUrl}/chat/completions` , `Authorization: Bearer <key>` , `response_format: {"type": "json_object"}` .

`baseUrl` defaults to `https://api.openai.com/v1` but can be overridden to point at any OpenAI-compatible endpoint. This is how the app supports free/local options without any OpenAI-specific code: OpenRouter, Groq, Ollama (`http://localhost:11434/v1`), LM Studio.

`AnthropicProvider`

Anthropic Messages API shape: `POST {baseUrl}/v1/messages` , headers `x-api-key` and `anthropic-version: 2023-06-01` , `system` + `messages` body. `baseUrl` defaults to `https://api.anthropic.com` and can be overridden for a proxy/gateway.

`provider_factory.dart`

```
AIProvider buildProvider(String providerName, String apiKey, String model, String? baseUrl)
```

The single registration point mapping a provider name string (`"openai"` / `"anthropic"`) to a concrete class. Unknown names throw `UnknownProviderError` .

`NoteEditorScreen` calls it through an optional `providerBuilder` constructor parameter that defaults to `buildProvider`. The app never sets it; widget tests pass a builder returning a fake provider, so the "Generate" flow is tested end to end without any HTTP.

Adding a new provider

1. Create `lib/ai/<provider>_provider.dart` with a class extending `AIProvider`, implementing `generateQuestions()`.
2. Reuse `buildQuestionGenerationPrompt()` from `prompts.dart` for the system/user prompt pair; don't write a new template unless the new provider's API genuinely can't use the same JSON-in/JSON-out contract.
3. Reuse `parseGeneratedQuestions()` from `base.dart` to parse the response.
4. Wrap every network/HTTP-status/JSON-parsing failure in `AIProviderError`.
5. Register the new class in `provider_factory.dart`.
6. Add the new provider name to the Account screen's provider dropdown (`lib/screens/account/account_screen.dart`).
7. Write tests mirroring `test/ai/openai_provider_test.dart`: mock every HTTP call with `http`'s `MockClient`, cover every error branch.

Where settings/keys live

- Non-secret settings (provider name, model, base URL, default question format): `lib/settings/app_settings.dart`, a JSON string in `shared_preferences` (which also works in the web build, unlike a file).
- The API key itself: `flutter_secure_storage`, written directly from the Account screen. There is no separate "secrets store" abstraction the way the Python desktop app has one (its `ApiKeyStore` interface exists because it needs to fall back to a plaintext file on systems with no OS keyring; `flutter_secure_storage` already handles that fallback internally per platform, so a second abstraction layer on top wasn't needed here).